Katedra počítačů ČVUT FEL & Czech ACM Chapter
Soutěž v programování 1995

Zadání úloh

Soutěž sponzorují **HEWLETT PACKARD** a **AutoCont**
a podporuje časopis **CHIP**

Problém A

Piškvorky

Snad každý zná velmi rozšířenou a zejména mezi školní mládeží populární hru piškvorky. Hra je určena pro dvojici hráčů. Hracím polem je obvykle čtverečkovaný papír. Každý z hráčů si vybere nějaký grafický symbol, většinou kolečko a křížek. Hráči pak střídavě umisťují své symboly do čtverečků v hracím plánu. Vítězem se stává ten hráč, který umístí určitý počet svých symbolů do těsně sousedících políček v řadě, sloupci či diagonále. Obvykle se hraje na pět takto umístěných symbolů.

Rádi bychom napsali program, který bude hrát piškvorky se svým lidským protějškem. Velmi důležitou částí tohoto programu je podprogram, který dokáže rozhodnout, jestli je situace na herním plánu již prohraná, nerozhodnutelná, nerozhodná či vede k jistému vítězství - vše za předpokladu, že ani jeden z hráčů neudělá žádnou chybu. Vaším úkolem je napsat takový podprogram.

Specifikace vstupu

Pro naše účely budeme používat hrací pole 4 x 4 čtverečky. Vítězem se stává ten hráč, kterému se podaří umístit čtyři své symboly vedle sebe ve sloupci, řádku či diagonále. Hráči mají symboly 'x' a 'o'. Počítač hraje se symbolem 'o'.

Každá úloha představuje jednu rozehranou partii piškvorek v situaci, kdy je počítač (symbol 'o') na tahu. Na prvním vstupním řádku je přirozené číslo N udávající počet úloh k vyřešení. Dále následují jednotlivé úlohy. Pro každou úlohu obsahuje vstup právě pět řádků. Z těchto pěti řádků první čtyři obsahují vždy právě čtyři znaky. Tyto znaky jsou 'x', 'o' nebo '.' (tečka). Znak 'x' označuje čtvereček, ve kterém má umístěn svůj symbol protihráč, znak 'o' označuje čtvereček patřící programu. Tečka označuje políčko, na kterém ani jeden hráč nemá svůj symbol. Pátý řádek je vždy prázdný - slouží pouze k oddělení jednotlivých úloh. K posouzení může být předložena i situace, která by nastala pouze v případě, že by některý hráč táhnul několikrát za sebou.

Specifikace výstupu

Pro každou úlohu obsahuje výstup právě jeden řádek, na kterém je jedno z následujících hlášení: 'Vyhra', 'Prohra', 'Remiza', 'Nerozhodnutelne'. Hlášení 'Vyhra' znamená, že tato partie je již vyhraná nebo vede pro počítač (tedy pro symbol 'o') k jistému vítězství za předpokladu, že počítač neudělá žádnou chybu. Hlášení 'Prohra' znamená, že tato partie je pro počítač (tedy pro symbol 'o') již prohraná nebo vede k jisté prohře za předpokladu, že protihráč neudělá žádnou chybu. Hlášení 'Remiza' znamená, že tato partie je remíza nebo vede zcela jistě k remíze bez jakýchkoliv dalších předpokladů. Hlášení 'Nerozhodnutelne' znamená, že nenastala ani jedna z předchozích situací. Situace, kde mají oba hráči již piškvorku, musí být označena jako 'Vyhra'.

Příklad vstupu

```
7
o..x
o...
o...
....

o..x
o..x
o..x
o..x

x..o
.ox.
.xo.
o..x

x..o
....
....
....

....
x..o
x..o
x..o

o.x.
.xox
xox.
xx.o

x...
....
.x..
x..x
```

Příklad výstupu

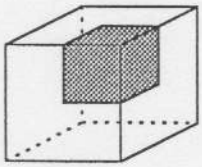
```
Vyhra
Vyhra
Remiza
Nerozhodnutelne
Vyhra
Remiza
Prohra
```

Problém B

Octree

Octree je struktura, používaná ve 3D grafice k ukládání informace o prostorovém objektu. Princip spočívá v tom, že se prostorová scéna uzavře do myšlené krychle a tato krychle v prostoru se podle svého obsahu označí buď jako plná, pokud objekt tuto krychli zcela vyplňuje, nebo jako prázdná, pokud v této krychli žádný objekt není, nebo jako zčásti zaplněná, pokud objekt v krychli leží, avšak nezaplňuje ji zcela. V takovém případě se provede rozdělení této částečně zaplněné krychle na 8 shodných menších krychlí a postup se pro ně opakuje. Například pokud budeme mít objekt, který vyplňuje krychličku o straně 1/2 strany základní krychle umístěnou v rohu této základní krychle podle obr. 1, bude popis vypadat takto:

Základní krychle je zčásti zaplněna
1 krychle je prázdná
2 krychle je prázdná
3 krychle je prázdná
4 krychle je prázdná
5 krychle je plná
6 krychle je prázdná
7 krychle je prázdná
8 krychle je prázdná



Obr. 1

Pro specifikaci nejmenšího bodu prostorové scény se požívá pojmu voxel (obdoba pojmu pixel ve 2D), což je nejmenší možná zcela zaplněná krychlička scény. Pro naše potřeby uvažujme krychli o straně 32 voxelů, celá krychle má tedy 32768 voxelů. Vaším úkolem bude napsat program, který bude mít jako vstup slovně popsanou octree strukturu této krychle a určí, kolik voxelů krychle je zaplněno. V našem příkladu na obr. 1 je zaplněno 16*16*16 = 4096 voxelů.

Specifikace vstupu

První řádek obsahuje přirozené číslo N udávající počet úloh k vyřešení. Dále následuje popis struktury N krychlí. Popis krychle začíná znakem '{' na samostatném řádku. Dále následuje jedno z klíčových slov:

FULL - pro krychli zcela zaplněnou
EMPTY - pro krychli zcela prázdnou
PARTIAL - pro krychli částečně zaplněnou

Všechna klíčová slova leží na samostatném řádku. Po slovech FULL a EMPTY následuje popis dalších krychlí, jsou-li ještě nějaké nepopsané. Po slově PARTIAL následuje na samostatném řádku znak '{' a popis osmi částí této krychle. Každá struktura uvozená znakem '{' na samostatném řádku je uzavřena znakem '}' na samostatném řádku. Na každém řádku může být libovolný počet mezer před slovem i za slovem. Vstup může obsahovat také prázdné řádky.

Specifikace výstupu

Pro každou testovanou úlohu obsahuje výstup právě jeden řádek s textem 'Scena obsahuje X voxelu.', kde X je přirozené číslo, které udává počet zaplněných voxelů.

Příklad vstupu

```
3
{
 FULL
}
{
 PARTIAL
 {
  FULL
  FULL
  EMPTY
  FULL
  EMPTY
  EMPTY
  FULL
  FULL
 }
}
{
 PARTIAL
 {
  FULL
  FULL
  FULL
  FULL
  FULL
  FULL
  FULL
  PARTIAL
  {
   EMPTY
   EMPTY
   EMPTY
   EMPTY
   EMPTY
   FULL
   FULL
   FULL
  }
 }
}
```

Příklad výstupu

```
Scena obsahuje 32768 voxelu.
Scena obsahuje 20480 voxelu.
Scena obsahuje 30208 voxelu.
```

Problém C

Šifra

Jeden egyptolog objevil uzamknutou část hrobky. Okolo hrobky bylo mnoho čtveřic přepínačů, o kterých se domníval, že slouží k odemknutí. Nad každou čtveřicí přepínačů bylo dlouhé číslo. Pokusy o násilné otevření hrobky ztroskotaly. Na pomoc si proto přizval kryptologa a společně po značném úsilí určili metodu, jak z čísla nad čtveřicí určit správné nastavení přepínačů. Stačí určit dělitelnost tohoto čísla čísly 5, 6, 7 a 8. První přepínač má být nastaven, je-li číslo nad přepínačem dělitelné 5, druhý je-li číslo dělitelné 6 atd. Vaším úkolem je napsat program, který pro zadané číslo určí správné nastavení čtveřice přepínačů.

Specifikace vstupu

První řádek obsahuje přirozené číslo N udávající počet příkladů k vyřešení. Dále následuje N řádků, na kterých jsou čísla délky L (0 < L <= 100). Každé číslo je na samostatném řádku.

Specifikace výstupu

Program dá N odpovědí. Pro každý příklad program odpoví jedním řádkem takto: 'Spravne nastaveni prepinacu je toto: d5 d6 d7 d8'; kde dX je buď 'Ano' nebo 'Ne' podle toho, jestli zadané číslo je dělitelné číslem X nebo není.

Příklad vstupu

```
5
96
35
70000000000000000
67
1
```

Příklad výstupu

```
Spravne nastaveni prepinacu je toto: Ne Ano Ne Ano
Spravne nastaveni prepinacu je toto: Ano Ne Ano Ne
Spravne nastaveni prepinacu je toto: Ano Ne Ano Ano
Spravne nastaveni prepinacu je toto: Ne Ne Ne Ne
Spravne nastaveni prepinacu je toto: Ne Ne Ne Ne
```

Problém D

Fibonacciho posloupnost

K řešení praktických technických problémů je téměř vždy třeba nějaký matematický aparát. K řešení určité třídy úloh je třeba pracovat s tzv. Fibonacciho posloupností. Je velice jednoduše popsána rekurentním vzorcem, ale vzorec pro přímý výpočet K-tého členu lze obtížně nalézt. Potřebuji-li znát hodnotu daného členu, většinou ji musím zjistit pomocí hrubé síly - počítačem. I když jsou dnešní počítače velice výkonné, nejdůležitější je vždy volba efektivního algoritmu. Vaším úkolem je dle níže uvedené specifikace nalézt vhodný algoritmus řešící popsaný problém.

Je dána posloupnost čísel F délky L předpisem:

F(0) = 0
F(1) = 1
F(i) = F(i-1) + F(i-2) pro i = 2..L-1

a dále číslo M a posloupnost G:

G(i) = F(i) mod M pro i = 0..L-1

Posloupnost G seřadíme vzestupně. Tuto seřazenou posloupnost nazveme G'. Otázka zní: Jaký je K-tý prvek této seřazené posloupnosti ?

Např. pro L = 6, M = 3 a K = 4 je
F = {0, 1, 1, 2, 3, 5}
G = {0, 1, 1, 2, 0, 2}
G' = {0, 0, 1, 1, 2, 2}
G'(4) = 2

Specifikace vstupu

První řádek obsahuje přirozené číslo N udávající počet úloh k vyřešení. Dále následuje N řádků. Na každém řádku se nachází trojice čísel určující jednu úlohu k vyřešení v pořadí L, M a K, které jsou odděleny mezerou. Velikosti jednotlivých vstupních čísel jsou z intervalů
L = 2..1 000 000 000
M = 2..200
K = 0..L-1

Specifikace výstupu

Pro každou úlohu musí výstup obsahovat právě jeden řádek s právě jedním číslem. Toto číslo je K-tým prvkem posloupnosti G'.

Příklad vstupu

```
4
6 3 4
11 5 8
1111 55 777
4321 42 1234
```

Příklad výstupu

```
2
3
21
5
```

Problém E

Vzdálenosti čísel

V této úloze si budeme hrát s čísly. Zkuste si vzít několik čísel (každé dvakrát) a zapsat je vedle sebe do řady tak, aby mezi šestkou a druhou šestkou bylo právě šest jiných čísel, mezi 42 a 42 bylo rovněž právě 42 jiných čísel atd. Jistě namítnete, že je to jednoduché, stačí brát jen sudá čísla:

```
8 6 4 2 0 0 2 4 6 8
```

Úkol je nalézt takovou posloupnost *bez* vynechávání některých čísel. Formálně lze žádanou posloupnost popsat takto:

- každé číslo se v posloupnosti vyskytuje právě dvakrát.
- obsahuje-li posloupnost číslo A>0, potom obsahuje také číslo A-1 (žádné číslo není vynecháno)
- mezi oběma výskyty čísla A je právě A čísel jiných: A(i) = A(j) => |i-j| -1 = A(i)

Specifikace vstupu

První řádek obsahuje přirozené číslo N udávající počet úloh k vyřešení. Dále následuje N řádků. Na každém řádku je vždy jedno číslo X z intervalu 0..1000 definující zadání (viz specifikace výstupu).

Specifikace výstupu

Pro každou úlohu musí výstup obsahovat právě jeden řádek, na němž bude zapsaná vygenerovaná posloupnost čísel oddělených navzájem mezerou. Posloupnost musí splňovat výše uvedená pravidla a navíc musí být tak dlouhá, aby obsahovala číslo X zadané jako vstupní parametr. Dlužno poznamenat, že řešení této úlohy není jednoznačné. Pro některé délky posloupnosti existuje více řešení. Program může vygenerovat i delší posloupnost než do čísla X. Za správné bude považováno libovolné řešení vyhovující výše uvedeným požadavkům a obsahující číslo X.

Příklad vstupu

```
3
0
5
6
```

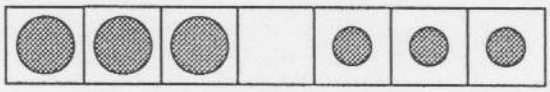
Příklad výstupu

```
0 0
4 1 7 1 6 4 2 5 3 2 7 6 3 5 0 0
7 5 3 1 6 1 3 5 7 2 4 6 2 0 0 4
```

Problém F

Mince

V této úloze se budeme opět zabývat hrou. Jde o hru s mincemi pro jednoho hráče, která má některé rysy společné se známou hrou 15. Hracími kameny je N desetníků a N pětníků. Hrací pole ma rozměr 2*N+1. Na počátku hry jsou desetníky na levé straně, pětníky na pravé a mezi nimi je jedno volné políčko. Pro N = 3 demonstruje výchozí postavení obrázek:



Mincemi je možné pohybovat podle následujících pravidel:

- desetník je možné přemístit o 1 políčko doprava, pokud je toto políčko volné
- desetníkem je možné přeskočit 1 minci vpravo, pokud je za ní volné políčko
- pětník je možné přemístit o 1 políčko doleva, pokud je toto políčko volné
- pětníkem je možné přeskočit 1 minci vlevo, pokud je za ní volné políčko

Cílem hry je přesunout všechny desetníky doprava a všechny pětníky doleva tak, že mezi nimi bude opět jedno políčko mezera. Vaším úkolem je napsat program, který umožní z již rozehrané hry určit, zda je možné ji z daného postavení dohrát.

Specifikace vstupu

První řádek obsahuje přirozené číslo N udávající počet úloh k vyřešení. Dále následuje N řádků. Na každém řádku je popsána jedna hra řetězcem znaků. V tomto řetězci znak 'd' označuje desetníky, znak 'p' pětníky a znak 'm' volné políčko.

Specifikace výstupu

Pro každou úlohu obsahuje výstup právě jeden řádek, na kterém je jedno z následujících hlášení:
'Hru lze ze zadaneho postaveni dohrat.'
'Hru nelze ze zadaneho postaveni dohrat.'

Příklad vstupu

```
5
dddmppp
mppppdddd
mddpp
ddpmdpp
dpdpdpm
```

Příklad výstupu

```
Hru lze ze zadaneho postaveni dohrat.
Hru lze ze zadaneho postaveni dohrat.
Hru nelze ze zadaneho postaveni dohrat.
Hru nelze ze zadaneho postaveni dohrat.
Hru lze ze zadaneho postaveni dohrat.
```

Problém G

Drak

Fraktální množiny můžeme jednoduše definovat jako množiny (bodů na obrazovce), jejichž části se podobají celku. Příkladem takového fraktálu je tzv. *drak*. Vznikne tak, že úzký proužek papíru přeložíme na polovinu, pak stejným způsobem opět na polovinu atd. Celkový počet přeložení nazveme úrovní. Přeložený proužek rozložíme tak, aby překladové hrany svíraly vždy úhel 90 stupňů a položíme takto rozložený proužek papíru na stůl. Pro úrovně 0 až 4 vypadá *drak* takto:



Úroveň 0 Úroveň 1 Úroveň 2 Úroveň 3 Úroveň 4

Úsečky takto vzniklé lomené čáry nazveme segmenty. Jednotlivé segmenty mají stejnou jednotkovou délku. Pro úroveň L bude tedy počet segmentů 2^L. Procházíme-li po této lomené čáře z počátku [0,0], pak každý segment má určitý směr a začíná na jisté souřadnici. Očíslujeme-li segmenty od nuly, pak segment číslo 2 začíná na souřadnici [1,1] a směřuje na jih. Vaším úkolem je napsat program, který pro zadané číslo segmentu určí souřadnice jeho začátku a směr, kterým segment směřuje. Nultý segment začíná na souřadnici [0,0] a směřuje na sever. Směry jsou značeny stejně jako obvykle na mapách tj. nahoru je sever, dolů jih, doleva západ a doprava východ.

Specifikace vstupu

První řádek obsahuje přirozené číslo N udávající počet úloh k vyřešení. Dále následuje N řádků. Na každém řádku je vždy jedno přirozené číslo z intervalu 0..1 000 000 000 udávající číslo segmentu, který nás zajímá.

Specifikace výstupu

Pro každou úlohu musí program vygenerovat právě jeden řádek, který obsahuje jeden znak směru daného segmentu, souřadnici X a souřadnici Y začátku tohoto segmentu navzájem oddělené mezerou. Znak určující směr segmentu může být N, E, S nebo W. N = sever (nahoru), E = východ (doprava), S = jih (dolu), W = západ (doleva).

Příklad vstupu

```
6
3
0
2
5
7
1995
```

Příklad výstupu

```
E 1 0
N 0 0
S 1 1
W 2 -1
E 1 -2
E 31 -38
```